

Univerzita Jana Evangelisty Purkyně
v Ústí nad Labem
Přírodovědecká fakulta



GoodAccess CLI klient pro Linux

BAKALÁŘSKÁ PRÁCE

Vypracoval: Valdemar Pospíšil

Vedoucí práce: Ing. Jiří Hromádka

Studijní program: Aplikovaná informatika

Studijní obor:

ÚSTÍ NAD LABEM 2026

Namísto žlutých stránek vložte digitálně podepsané zadání kvalifikační práce poskytnuté vedoucím katedry.

Zadání musí zaujímat právě dvě strany.

Zadání je nutno vložit jako PDF pomocí některého nástroje, který umožňuje editaci dokumentů (se zachováním elektronického podpisu).

V Linuxe lze například použít příkaz `pdftk`.

Prohlášení

Prohlašuji, že jsem tuto bakalářskou práci vypracoval samostatně a použil jen pramenů, které cituji a uvádím v přiloženém seznamu literatury.

Byl jsem seznámen s tím, že se na moji práci vztahují práva a povinnosti vyplývající ze zákona č. 121/2000 Sb., ve znění zákona č. 81/2005 Sb., autorský zákon, zejména se skutečností, že Univerzita Jana Evangelisty Purkyně v Ústí nad Labem má právo na uzavření licenční smlouvy o užití této práce jako školního díla podle § 60 odst. 1 autorského zákona, a s tím, že pokud dojde k užití této práce mnou nebo bude poskytnuta licence o užití jinému subjektu, je Univerzita Jana Evangelisty Purkyně v Ústí nad Labem oprávněna ode mne požadovat přiměřený příspěvek na úhradu nákladu, které na vytvoření díla vynaložila, a to podle okolností až do jejich skutečné výše.

V Ústí nad Labem dne 6. března 2026

Podpis:

Děkuji vedoucímu práce **Ing. Jiřímu Hromádkovi**
za neocenitelné rady a pomoc při tvorbě bakalářské práce.

Abstrakt:

V současné době nabízí GoodAccess VPN řešení především prostřednictvím grafického uživatelského rozhraní, což nevyhovuje všem uživatelským scénářům. Zejména v prostředí serverů, automatizovaných systémů a DevOps pracovních postupů preferují administrátoři a pokročilí uživatelé řešení příkazové řádky.

Hlavním cílem této práce je navrhnout a implementovat funkční prototyp CLI aplikace pro správu GoodAccess VPN připojení na Linux distribucích s využitím WireGuard protokolu. Práce se zabývá návrhem architektury, bezpečným ukládáním dat a implementací komunikace se systémovou službou pomocí IPC.

Klíčová slova: VPN, GoodAccess, CLI, Linux, Go, .NET, WireGuard, IPC, Clean Architecture

GOODACCESS CLI CLIENT FOR LINUX

Abstract:

Currently, GoodAccess VPN offers solutions primarily through a graphical user interface, which does not suit all user scenarios. Especially in server environments, automated systems, and DevOps workflows, administrators and advanced users prefer command-line solutions.

The main goal of this work is to design and implement a functional prototype of a CLI application for managing GoodAccess VPN connections on Linux distributions using the WireGuard protocol. The work deals with the design of the architecture, secure data storage, and implementation of communication with the system service using IPC.

Keywords: VPN, GoodAccess, CLI, Linux, Go, .NET, WireGuard, IPC, Clean Architecture

Obsah

Úvod	15
1. Teoretická východiska	17
1.1. Rozhraní příkazové řádky	17
1.2. Architektura VPN a principy Zero Trust	19
1.3. Architektura operačního systému Linux	21
1.4. Technologie a nástroje	24
1.5. Zajištění kvality a testování softwaru	27
1.6. Distribuce softwaru a balíčkovací systémy	29
1.7. Agilní metodiky a Extreme Programming	31
2. Analýza a specifikace požadavků	33
2.1. Identifikace zúčastněných stran	33
2.2. Případy užití	33
2.3. Funkční požadavky	33
2.4. Nefunkční požadavky	35
2.5. Analýza architektury	36
3. Návrh řešení	39
3.1. Celková architektura systému	39
3.2. Návrh komunikace (IPC)	40
3.3. Návrh uživatelského rozhraní (CLI)	40
3.4. Bezpečnostní model	40
3.5. Datový model a konfigurace	40
4. Implementace	43
4.1. Struktura projektu a nástroje	43
4.2. Implementace klienta (Frontend)	43
4.3. Úpravy backendové služby	43
4.4. Integrace WireGuardu	43
4.5. Distribuční balíčky a Systemd	43
5. Testování a validace	45
5.1. Metodika testování	45
5.2. Jednotkové testy (Unit Testing)	45

5.3. Integrační a systémové testy	45
5.4. Testování na různých distribucích	45
Závěr	47
A. Externí přílohy	51
B. Další přílohy	53

Seznam použitých zkratk

API Application Programming Interface

APT Advanced Package Tool

BDD Behavior-Driven Development

CI Continuous Integration

CLI Command Line Interface

DAC Discretionary Access Control

E2E End-to-End

FHS Filesystem Hierarchy Standard

GUI Graphical User Interface

IPC Inter-Process Communication

JSON JavaScript Object Notation

OIDC OpenID Connect

PID Process Identifier

POSIX Portable Operating System Interface

QA Quality Assurance

SDLC Software Development Life Cycle

SDP Software Defined Perimeter

SSH Secure Shell

SSO Single Sign-On

TDD Test-Driven Development

TUI Text User Interface

UDS Unix Domain Sockets

UID User Identifier

VPN Virtual Private Network

XP Extreme Programming

ZTNA Zero Trust Network Access

Úvod

V posledních letech došlo k zásadnímu posunu v organizaci práce a správě síťové infrastruktury. S masivním přechodem na práci na dálku a distribuované týmy se tradiční modely zabezpečení sítě ukazují jako nedostatečné. Organizace stále častěji adoptují architektury typu Zero Trust Network Access (ZTNA), kde je bezpečné VPN připojení klíčovým prvkem nejen pro koncové uživatele, ale i pro serverovou infrastrukturu.

Platforma GoodAccess v současné době nabízí řešení primárně prostřednictvím grafického uživatelského rozhraní (GUI). Toto řešení je vhodné pro běžné uživatele na osobních počítačích, avšak naráží na limity v prostředí serverů, automatizovaných systémů a DevOps pracovních postupů.

Motivace

V serverovém prostředí Linux, které je dominantní platformou pro cloudovou infrastrukturu, často není grafické rozhraní dostupné (tzv. headless servery). Administrátoři a DevOps inženýři vyžadují nástroje, které lze ovládat z příkazové řádky, snadno skriptovat a integrovat do CI/CD pipelin. Absence nativního CLI (Command Line Interface) klienta pro Linux představuje pro GoodAccess značné omezení v možnosti nasazení v těchto scénářích.

Cíle práce

Hlavním cílem této bakalářské práce je navrhnout a implementovat funkční prototyp CLI aplikace pro správu GoodAccess VPN připojení na Linux distribucích. Důraz je kladen na vytvoření "lehkého" a nativního uživatelského rozhraní, které efektivně komunikuje se systémovou službou a využívá stávající podnikovou logiku pro správu VPN tunelů. Práce má ambici ukázat kompletní inženýrský proces vývoje softwaru od analýzy požadavků až po distribuci finálních balíčků.

Dílčí cíle práce zahrnují:

- Analýzu a návrh architektury založené na principu oddělení logiky od prezentace (tzv. "stupid frontend" přístup).
- Implementaci meziprocesní komunikace (IPC) mezi uživatelským rozhraním v jazyce Go a systémovým modulem v .NET prostřednictvím Unix Domain Sockets.

- Vytvoření modulu v .NET pro bezpečné řízení VPN tunelů (OpenVPN, WireGuard) s využitím existujících komponent platformy GoodAccess.
- Návrh a realizaci automatizovaného testování a balíčkování aplikace pro distribuce Debian a Fedora.

Struktura práce

Práce je rozdělena do několika logických celků. Kapitola 1 se věnuje teoretickým východiskům, popisuje architekturu Linuxu, použité VPN protokoly a metody meziprocesové komunikace. Kapitola 2 definuje požadavky na systém a analyzuje možné architektonické přístupy. Následující kapitoly se poté věnují samotnému návrhu, implementaci a testování výsledného řešení.

1. Teoretická východiska

Tato kapitola analyzuje klíčové technologie, architektonické principy a bezpečnostní standardy nezbytné pro návrh moderních systémových nástrojů v prostředí OS Linux. Text postupuje od obecné problematiky příkazové řádky přes síťové protokoly až po specifika implementačních platforem .NET a Go.

1.1. Rozhraní příkazové řádky

Rozhraní příkazové řádky (CLI – Command Line Interface) představuje jeden z nejstarších, avšak stále nejvýkonnějších způsobů interakce uživatele s počítačem. Navzdory masivnímu rozšíření grafických uživatelských rozhraní (GUI) v 90. letech 20. století zůstává CLI nepostradatelným nástrojem pro systémové administrátory, vývojáře a pokročilé uživatele, zejména v prostředí operačních systémů typu Unix a Linux.

Role CLI v moderní správě systémů

Efektivita a automatizace

Hlavním důvodem preference příkazové řádky v profesionálním prostředí je efektivita, rychlost a snadná automatizace. Zatímco grafické rozhraní je navrženo pro intuitivní objevování funkcí pomocí vizuálních prvků, CLI se zaměřuje na precizní a opakovatelné provádění úkolů. Administrátor může pomocí jediného příkazu nebo krátkého skriptu provést operaci na stovkách serverů současně, což je v rámci GUI prakticky neproveditelné [1].

Vzdálená správa a cloud

Dalším kritickým faktorem je vzdálená správa. Protokol SSH (Secure Shell), který je de facto standardem pro správu linuxových serverů, je primárně textově orientovaný. Přenos textových příkazů a jejich výstupů vyžaduje minimální síťovou propustnost, což umožňuje spolehlivou správu systémů i přes nestabilní nebo pomalé připojení [2]. V prostředí cloudu a kontejnerizace (např. Docker, Kubernetes), kde systémy často nemají žádné grafické výstupní zařízení (headless režim), je CLI jedinou možností interakce.

Principy návrhu a standardy CLI aplikací

Kvalitní CLI aplikace se řídí souborem ustálených principů, které zajišťují jejich předvídatelnost a vzájemnou spolupráci. Eric S. Raymond ve své knize *The Art of Unix Programming* definuje tzv. „Unixovou filosofii“, jejímž základem je tvorba malých, jednoúčelových nástrojů, které spolu efektivně komunikují [1]. Tento přístup umožňuje skládat komplexní operace z jednoduchých komponent pomocí mechanismu rour (pipes).

Unixová filozofie

Základním stavebním kamenem interakce v CLI jsou standardní datové proudy, které umožňují řetězení příkazů do složitých celků [1]:

- **Standardní vstup (stdin):** Zdroj dat pro aplikaci, typicky klávesnice nebo výstup jiného programu.
- **Standardní výstup (stdout):** Primární kanál pro výstup dat, typicky terminál.
- **Standardní chybový výstup (stderr):** Oddělený kanál pro chybová hlášení a diagnostiku, což umožňuje uživateli přeměrovat data do souboru, zatímco chyby stále vidí na obrazovce.

Standardní datové proudy

Důležitou roli hraje také syntaxe příkazů. Moderní nástroje se snaží dodržovat standardy jako POSIX nebo GNU konvence pro argumenty a přepínače (flags). Krátké přepínače (např. `-v`) jsou určeny pro rychlé psaní, zatímco dlouhé varianty (např. `--verbose`) zvyšují čitelnost skriptů a dokumentace [1].

Syntaxe a standardy

CLI versus TUI

V rámci terminálu se můžeme setkat také s textovým uživatelským rozhraním (TUI – Text User Interface). Na rozdíl od klasického CLI, které vypisuje řádky textu v sekvenčním pořadí, TUI využívá celou plochu terminálu k vykreslování vizuálních prvků, jako jsou okna, menu, tabulky nebo interaktivní grafy, často s využitím knihoven jako `ncurses` nebo moderních frameworků v jazyce Go (např. `Bubble Tea` [3]).

Hlavní rozdíl spočívá v komplexitě a způsobu interakce. CLI je ideální pro jednorázové příkazy a automatizaci v nekonečném proudu (batch processing). TUI je naproti tomu vhodnější pro interaktivní nástroje, kde uživatel potřebuje neustálý přehled o stavu systému nebo kde je navigace v datech pomocí klávesových zkratk efektivnější než vypisování parametrů (např. textové editory, správci souborů nebo dashboardy pro monitoring).

1.2. Architektura VPN a principy Zero Trust

Tradiční pojetí zabezpečení sítě, založené na ochraně perimetru, prochází v posledních letech zásadní transformací. Tato sekce popisuje evoluci od klasických VPN tunelů k moderní architektuře Zero Trust Network Access (ZTNA).

Tradiční VPN a perimetrová bezpečnost

Tunelování a enkapsulace

Virtuální privátní síť (VPN) je technologie, která umožňuje vytvořit bezpečné, šifrované spojení (tunel) mezi dvěma body v rámci nezabezpečené sítě, jako je internet. Princip tunelování spočívá v zapouzdření (enkapsulaci) síťových paketů do jiného protokolu, který zajišťuje integritu a důvěrnost přenášených dat [4].

Model hrad a příkop

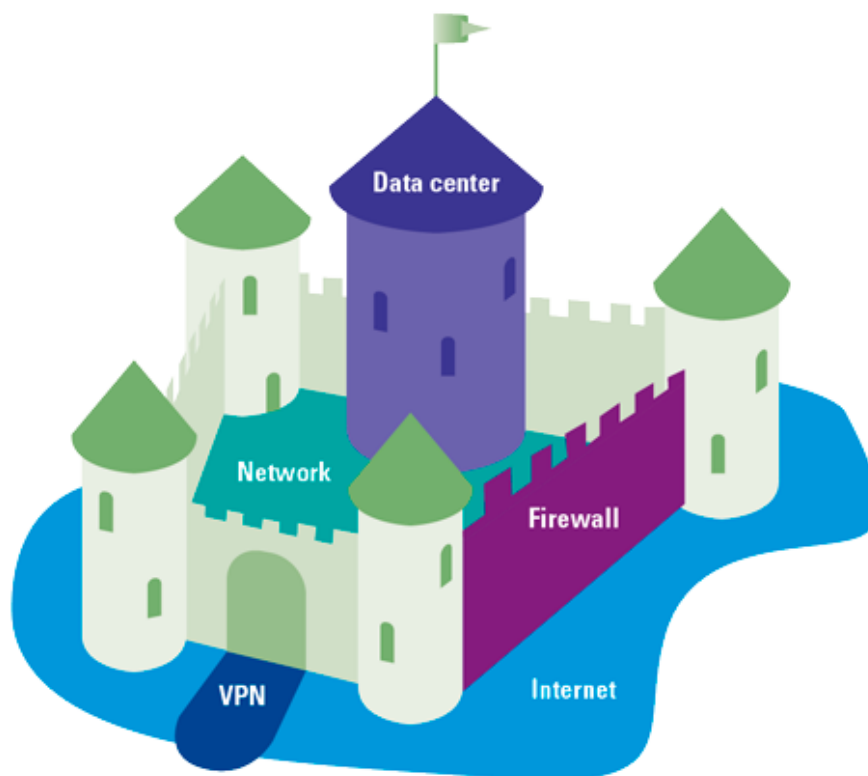
Historicky se organizace spoléhaly na model „hrad a příkop“ (Castle-and-Moat). V tomto modelu je vnitřní síť považována za důvěryhodnou zónu, která je od vnějšího světa oddělena firewallem (příkopem) [5]. VPN v tomto kontextu slouží jako „padací most“, který umožňuje vzdáleným uživatelům vstoupit do vnitřní sítě. Jakmile však uživatel získá přístup, má často široký dosah k mnoha zdrojům v rámci sítě. Tento přístup trpí kritickou slabinou: pokud útočník kompromituje jediné zařízení uvnitř perimetru, může se v síti volně pohybovat (laterální pohyb) a napadat další systémy, protože vnitřní síť mu implicitně důvěřuje [6].

Koncept Zero Trust

Filozofie Zero Trust (nulová důvěra) model „hrad a příkop“ opouští a vychází z principu „nikdy nedůvěřuj, vždy prověřuj“ (Never trust, always verify). V konceptu Zero Trust není důvěra nikdy udělena automaticky na základě fyzického umístění uživatele nebo jeho IP adresy. Každý pokus o přístup k jakémukoliv zdroji musí být individuálně autentizován, autorizován a šifrován.

Tabulka 1.1.: Srovnání modelů Castle-and-Moat a Zero Trust [7]

Kritérium	Castle-and-Moat	Zero Trust
Filozofie	„Důvěřuj, ale prověřuj“	„Nikdy nedůvěřuj, vždy prověřuj“
Perimetr	Statický (firewall)	Dynamický (identita uživatele)
Úroveň přístupu	Široký přístup k síti	Granulární (Least Privilege)
Ověřování	Jednorázové (při vstupu)	Kontinuální (při každém požadavku)
Laterální pohyb	Snadný po průniku	Výrazně omezen segmentací



Obrázek 1.1.: Vizualizace modelu Castle-and-Moat [5]

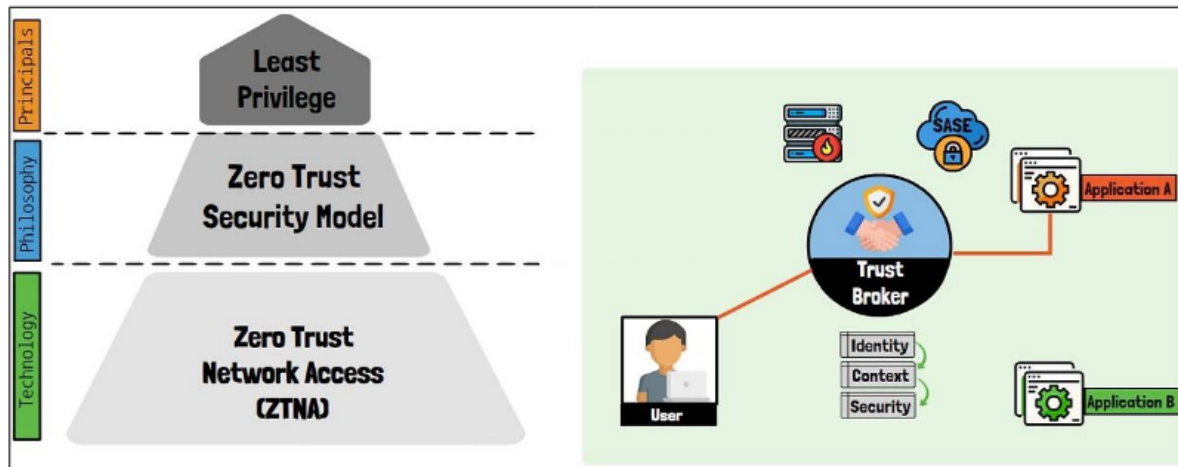
Zero Trust Network Access (ZTNA) a GoodAccess

Zero Trust Network Access (ZTNA) představuje praktickou technologickou implementaci architektury Zero Trust. V architektuře ZTNA je kontrola přístupu striktně vynucována pro každou relaci.

Klíčovým prvkem ZTNA je Software Defined Perimeter (SDP). SDP vytváří dynamickou, virtuální hranici kolem aplikací a služeb, čímž je činí „neviditelnými“ pro veřejný internet i pro neautorizované uživatele v lokální síti. GoodAccess implementuje tyto principy formou cloudové platformy, která nahrazuje tradiční VPN brány [8]. Místo statických IP adres využívá identitu uživatele (často propojenou s SSO/OIDC) a kontext zařízení (tzv. Device Posture) k dynamickému přidělování přístupových práv k jednotlivým systémům [9]. Tím se radikálně snižuje plocha možného útoku, protože uživatel vidí pouze ty zdroje, ke kterým má explicitní oprávnění.

Protokol WireGuard

WireGuard je moderní VPN protokol navržený s důrazem na jednoduchost, rychlost a moderní kryptografii [4]. S rozsahem přibližně 4 000 řádků kódu je WireGuard velmi snadno auditovatelný. Díky své implementaci přímo v jádře Linuxu dosahuje vynikající propustnosti a velmi nízké latence. Z pohledu správy systému se WireGuard chová jako standardní síťové rozhraní, což výrazně zjednodušuje konfiguraci směrování a celkovou integraci do systémo-



Obrázek 1.2.: Architektura Zero Trust Network Access [10]

vých nástrojů. Pro dynamické enterprise prostředí, které GoodAccess obsluhuje, představuje WireGuard ideální volbu především díky mimořádně rychlému navazování spojení (handshake) a vysoké odolnosti vůči změnám síťového prostředí (roaming).

1.3. Architektura operačního systému Linux

Tato sekce analyzuje klíčové mechanismy operačního systému Linux, které jsou nezbytné pro běh systémových služeb a správu síťových rozhraní.

Struktura systému: User Space a Kernel Space

Linux dělí paměťový prostor na dvě privilegovaně odlišné části. Toto rozdělení je kritické pro stabilitu a bezpečnost systému.

Prostor jádra a uživatelský prostor

- **Kernel Space (Prostor jádra):** Zde běží jádro operačního systému, ovladače zařízení a kritické síťové moduly, včetně implementace protokolu WireGuard. Kód v tomto prostoru má plný přístup k hardwaru.
- **User Space (Uživatelský prostor):** Zde běží běžné aplikace, včetně uživatelských rozhraní a backendových služeb.

Toto oddělení, často označované jako *privilege separation*, zajišťuje, že chybný nebo škodlivý kód v uživatelské aplikaci nemůže přímo ohrozit integritu celého systému. Brian Ward ve své knize *How Linux Works* zdůrazňuje, že zatímco uživatelský prostor je místem, kde probíhá většina „reálné akce“ aplikací, jádro slouží jako arbitr a správce všech hardwarových prostředků [2].

Systémová volání a přepnutí kontextu

Aplikace v uživatelském prostoru nemohou přímo přistupovat k hardwaru. Musí využívat tzv. *systémová volání* (syscalls) pro komunikaci s jádrem. Přejed mezi těmito prostory vyžaduje tzv. *přepnutí kontextu* (context switch), což je operace s nenulovou režii. V kontextu VPN řešení je toto kritické při zpracování síťového provozu – například protokoly běžící v uživatelském prostoru musí každý paket kopírovat mezi prostory přes virtuální síťové rozhraní (TUN/TAP), což může omezovat propustnost při vysokém zatížení. Naopak implementace přímo v prostoru jádra tuto režii minimalizují a umožňují efektivnější zpracování datových toků [4].

Správa procesů a životní cyklus služeb

Proces je v Linuxu definován jako instance běžícího programu v paměti. Každý proces má svůj unikátní identifikátor (PID) a je spravován jádrem, které mu přiděluje procesorový čas a paměťové prostředky.

Procesy a daemony

Vytváření nových procesů probíhá pomocí dvojice systémových volání `fork()` a `exec()`. Volání `fork()` vytvoří identickou kopii rodičovského procesu, zatímco `exec()` nahradí kód běžícího procesu novým programem. Tento mechanismus je základem pro spouštění všech aplikací v systému [2].

Specifickou kategorií procesů jsou *daemony*. Jedná se o procesy, které běží na pozadí, nejsou přímo spojeny s žádným terminálem a typicky čekají na určité události nebo poskytují systémové služby. V moderní architektuře systémových nástrojů bývá logika implementována právě jako daemon, který udržuje stav (např. VPN spojení) nezávisle na tom, zda je uživatelské rozhraní zrovna spuštěno. Tento přístup zajišťuje persistenci spojení a umožňuje bezpečné provádění privilegovaných operací (např. manipulaci se síťovými rozhraními) pod identitou uživatele s dostatečnými právy, zatímco samotné rozhraní může běžet v kontextu běžného uživatele.

Správa služeb a systémová inicializace (systemd)

Moderní linuxové distribuce využívají pro správu systému a služeb inicializační systém `systemd`. Ten v posledním desetiletí nahradil starší standardy jako `sysvinit` a přinesl zásadní změny v tom, jak jsou procesy na pozadí spouštěny, monitorovány a hierarchicky organizovány.

systemd a správa služeb

Zatímco starší systémy spouštěly služby sekvenčně pomocí shell skriptů, což prodlužovalo start systému, **systemd** využívá agresivní paralelizaci a spouštění služeb na základě požadavků (on-demand), například skrze socketovou aktivaci. Brian Ward vysvětluje, že **systemd** není pouhým inicializačním procesem s PID 1, ale komplexním ekosystémem pro správu uživatelského prostoru, který integruje logování (**journald**), správu zařízení (**udev**) i konfiguraci sítě [2].

Z pohledu vývoje systémových nástrojů jsou klíčové tzv. *Unit files* (jednotky), konkrétně **.service** soubory. Tyto deklarativní konfigurační soubory definují parametry spuštění služby, její závislosti na síťové konektivitě a bezpečnostní kontext. Díky integraci s mechanismem *cgroups* (Control Groups) má **systemd** absolutní kontrolu nad všemi procesy patřícími k dané službě, což umožňuje jejich spolehlivé ukončení nebo automatický restart v případě havárie. V architektuře postavené na službách hraje **systemd** roli garanta běhu, zajišťuje spuštění po startu systému a poskytuje standardizované rozhraní pro monitoring stavu skrze nástroj **systemctl**.

Konfigurace síťového subsystému (Netlink)

Tradiční unixové nástroje ('ifconfig') byly v Linuxu nahrazeny modernějším balíkem 'iproute2'. Na pozadí těchto nástrojů stojí rozhraní **Netlink**.

Netlink slouží jako komunikační sběrnice mezi jádrem a uživatelským prostorem [2]. Na rozdíl od 'ioctl' volání, Netlink používá standardní mechanismus socketů pro předávání zpráv o konfiguraci sítě (např. přiřazení IP adresy, změna routovací tabulky).

Meziprocesová komunikace (IPC) v Linuxu

Jelikož je aplikace rozdělena na dvě části (privilegovaná služba a neprivilegovaný klient), je nutné zajistit efektivní přenos dat mezi nimi. V prostředí Linuxu je standardem pro lokální komunikaci využití **Unix Domain Sockets (UDS)**.

Unix Domain Sockets

Na rozdíl od TCP/IP socketů, které využívají síťová rozhraní a IP adresy, UDS využívají souborový systém. Socket je reprezentován speciálním souborem na disku [2]. Hlavní výhody pro systémové služby jsou:

1. **Výkon:** Data se nekopírují přes síťový zásobník, ale zůstávají v paměti OS.
2. **Bezpečnost a řízení přístupu:** Přístup k socketu lze omezit standardními linuxovými oprávněními souborů. To umožňuje, aby se k chráněné službě mohl připojit pouze uživatel s příslušným oprávněním.

Správa oprávnění a souborový systém

Linux využívá standard **FHS** (**F**ilesystem **H**ierarchy **S**tandard), který definuje, kam mají aplikace ukládat svá data. Pro software třetích stran je vyhrazen adresář `/opt`, zatímco konfigurace a logy patří do `/etc` a `/var/log`.

Model oprávnění DAC a bezpečnostní kontext

Linux využívá model Discretionary Access Control (DAC), kde každý soubor má vlastníka, skupinu a sadu práv (**rw**x) [2]. Pro systémové služby jako VPN klient je však tento model často nedostačující, protože operace se síťovým rozhraním vyžadují práva uživatele **root**.

V rámci bezpečného návrhu běží kritické komponenty pod identitou **root** (typicky jako `systemd` služba), zatímco frontend komunikuje přes Unix Domain Socket s omezeným přístupem. Moderní Linux dále rozšiřuje tento model o tzv. *Capabilities* (schopnosti), které umožňují rozdělit absolutní moc uživatele **root** na menší celky (např. `CAP_NET_ADMIN` pro správu sítě), čímž implementují princip nejmenších privilegií [2].

1.4. Technologie a nástroje

V rámci teoretického základu pro vývoj moderních systémových nástrojů je nezbytné analyzovat vlastnosti platform a technologií, které jsou pro tento typ úloh optimalizovány. Tato sekce se zaměřuje na platformu .NET 8, jazyk Go a transportní formáty dat, přičemž zkoumá jejich vnitřní architekturu a bezpečnostní mechanismy.

Platforma .NET 8

Platforma .NET 8 (dříve .NET Core) představuje moderní, multiplatformní vývojový ekosystém navržený pro vysoký výkon a škálovatelnost. Jedním z klíčových aspektů této platformy je její schopnost nativního běhu v prostředí OS Linux, což z ní činí vhodný nástroj pro vývoj serverových aplikací a systémových služeb [11].

Generic Host a životní cyklus služeb

Základním architektonickým prvkem pro běh aplikací na pozadí je tzv. **Generic Host**. Tento vzor poskytuje standardizované rozhraní pro správu životního cyklu aplikace, konfiguraci, logování a delegování úloh skrze mechanismus *Dependency Injection* (DI) [12]. Generic Host sjednocuje způsob, jakým jsou spravovány různé typy služeb, od webových serverů až po jednoduché úlohy běžící na pozadí. V kontextu systémových služeb umožňuje precizní řízení procesu ukončení (*graceful shutdown*), kdy aplikace při přijetí signálu k ukončení (např. `SIGTERM`) korektně uvolní prostředky a ukončí běžící operace, čímž předchází nekonzistenci dat.

Integrace se systémem systemd

Pro hlubší integraci s linuxovým inicializačním systémem **systemd** poskytuje platforma specializovaný modul **Microsoft.Extensions.Hosting.Systemd** [12]. Tato komponenta umožňuje .NET aplikaci nativně komunikovat s inicializačním systémem **systemd**. Díky této integraci může služba odesílat stavová hlášení (např. o úspěšném spuštění) přímo do **systemd** a využívat pokročilé funkce správy služeb, jako je automatické obnovení po havárii nebo logování skrze systémový žurnál (**journald**).

ASP.NET Core Data Protection

Kritickou součástí moderních systémů je ochrana citlivých dat, kterou v ekosystému .NET zajišťuje stack **ASP.NET Core Data Protection** [13]. Tento mechanismus je navržen k šifrování dat „v klidu“ (encryption at rest) a k bezpečné správě kryptografických klíčů. Architektura stacku je založena na hierarchickém systému klíčů, kde je každý datový prvek šifrován unikátním klíčem odvozeným z hlavního klíče (master key).

Bezpečnostní stack řeší několik klíčových problémů automatizovaně:

- **Izolace:** Klíče jsou vázány na konkrétní aplikaci, což zabraňuje dešifrování dat jiným procesem i na stejném systému.
- **Rotace:** Systém automaticky generuje nové klíče a zneplatňuje staré, čímž omezuje dopad případného úniku klíče.
- **Perzistence:** Na systému Linux stack podporuje ukládání klíčů ve formátu XML (Extensible Markup Language) do souborového systému, přičemž je možné využít šifrování klíčů pomocí certifikátů nebo systémových úložišť.

Díky této abstrakci může vývojář pracovat s jednoduchým rozhraním **IDataProtector**, zatímco platforma transparentně zajišťuje kryptografickou integritu a důvěrnosti uložených dat.

Programovací jazyk Go

Jazyk Go, vyvinutý společností Google, se stal standardem pro systémové programování a cloud-native nástroje. Jeho filozofie je založena na minimalismu, efektivitě a bezpečnosti. Go kombinuje výkon kompilovaných jazyků (jako C++) s produktivitou jazyků s dynamickou typovou kontrolou [14].

Charakteristika jazyka a systémové programování

Hlavní předností Go v oblasti systémových nástrojů je jeho schopnost statické kompilace. Výsledkem procesu sestavení je jediný binární soubor, který v sobě obsahuje všechny nezbytné knihovny (včetně runtime prostředí). To je v prostředí Linuxu zásadní výhoda, neboť

uživatel nemusí instalovat žádné závislosti, což radikálně zjednodušuje distribuci softwaru [15]. Go dále disponuje pokročilým modelem souběžnosti založeným na *gorutinách* a kanálech (*channels*), což umožňuje efektivní asynchronní zpracování síťových operací a událostí bez složitosti tradičních vláken.

Framework Cobra

Pro tvorbu rozhraní příkazové řádky (CLI) se v ekosystému Go prosadil framework **Cobra** [16]. Ten definuje standardizovanou strukturu pro definici příkazů, podrízených příkazů a jejich parametrů (včetně rozlišení mezi globálními a lokálními přepínači). Cobra je de facto standardem, na kterém staví nástroje jako Docker či Kubernetes. Framework automatizuje generování nápovědy, validaci argumentů a poskytuje rozhraní pro generování manuálových stránek nebo skriptů pro automatické doplňování v shellu, čímž zajišťuje konzistentní uživatelský zážitek.

Architektura Elm a framework Bubble Tea

V oblasti interaktivních textových rozhraní (TUI) představuje špičku framework **Bubble Tea** [3]. Ten je unikátní tím, že do terminálového prostředí přináší principy funkcionálního programování skrze vzor **The Elm Architecture** [17]. Tato architektura definuje tři základní komponenty:

1. **Model:** Datová struktura reprezentující aktuální stav aplikace.
2. **Update:** Čistá funkce, která přijímá zprávu (*Message*) a aktuální model a vrací model nový společně s případnými příkazy (*Commands*).
3. **View:** Funkce, která na základě modelu vygeneruje textovou reprezentaci rozhraní pro terminál.

Zásadním přínosem tohoto přístupu je determinismus a snadná testovatelnost. Veškeré změny stavu probíhají asynchronně skrze zasílání zpráv, což zabraňuje vzniku *race conditions*. Framework Bubble Tea se stará o efektivní překreslování terminálu (tzv. *diffing*), kdy jsou aktualizovány pouze ty části obrazovky, které se skutečně změnily. To umožňuje vytvářet plynulá a reaktivní rozhraní, která uživateli poskytují okamžitou zpětnou vazbu bez problikávání či zpoždění.

Serializace dat a transportní formáty

Efektivní výměna dat mezi procesy (IPC) vyžaduje volbu vhodného formátu pro serializaci, který balancuje mezi výkonem, čitelností a snadnou údržbou.

Formát JSON

JSON (JavaScript Object Notation) je široce rozšířený, textově orientovaný formát [18]. Jeho hlavní výhodou je transparentnost – přenášená data lze snadno monitorovat a analyzovat běžnými systémovými nástroji. JSON nevyžaduje definici schématu předem, což usnadňuje agilní vývoj a integraci s webovými API. Navzdory textové povaze je parsování moderními knihovnamí extrémně rychlé a pro většinu IPC scénářů na jednom hostiteli je režie zanedbatelná.

Protokol gRPC a Protocol Buffers

Naproti tomu **gRPC** je binární framework využívající **Protocol Buffers** jako mechanismus pro definici dat (*Interface Definition Language*) [19]. Je navržen pro maximální propustnost a minimální režii, čehož dosahuje díky binárnímu kódování a generování silně typovaného kódu pro různé programovací jazyky. gRPC vyniká v systémech s vysokým zatížením a v prostředí mikroslužeb, kde redukce velikosti zpráv vede k úspoře síťových prostředků.

Srovnání a volba transportní vrstvy

Srovnání těchto technologií ukazuje na odlišné priority. gRPC dominuje v oblasti čistého výkonu a typové bezpečnosti, vyžaduje však složitější proces sestavení aplikace a ztěžuje manuální inspekci dat [19]. JSON naopak zůstává preferovanou volbou pro systémy, kde je kladen důraz na diagnostiku a jednoduchost implementace.

Tabulka 1.2.: Srovnání formátů JSON a gRPC [19]

Kritérium	JSON	gRPC (Protobuf)
Typ dat	Textový (UTF-8)	Binární
Schéma	Volitelné	Povinné (.proto)
Výkon	Nižší (parsování textu)	Vysoký (binární serializace)
Čitelnost	Výborná pro člověka	Vyžaduje dekodér
Podpora prohlížečů	Nativní	Vyžaduje proxy

V rámci lokální komunikace mezi systémovými nástroji, kde je důležitá možnost nahlédnout do komunikace mezi klientem a službou bez nutnosti binární dešifrace, představuje JSON optimální kompromis mezi technickou efektivitou a vývojovou udržitelností.

1.5. Zajištění kvality a testování softwaru

Kvalita softwaru není náhodným výsledkem, ale důsledkem systematického procesu, který doprovází celý životní cyklus vývoje (SDLC – Software Development Life Cycle). V moderním inženýrství, zejména u systémových nástrojů, které manipulují se síťovým nastavením

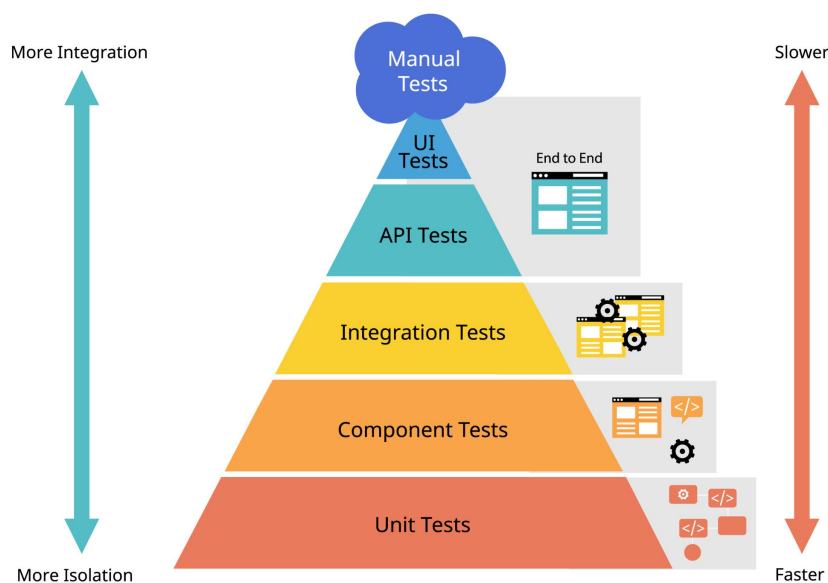
a vyžadují vysoká oprávnění, je verifikace správnosti klíčovým prvkem pro zajištění stability a bezpečnosti systému.

Kvalita softwaru v SDLC

Tradiční pohled na testování jako na oddělenou fázi na konci vývoje je v agilním prostředí nahrazen integrací testů přímo do procesu tvorby kódu. Zajištění kvality (QA – Quality Assurance) zahrnuje nejen detekci chyb, ale především prevenci jejich vzniku skrze statickou analýzu, code review a automatizované testování [20]. Automatizace v tomto procesu umožňuje rychlou regresní analýzu při jakýchkoliv změnách v systému.

Testovací pyramida

Koncept testovací pyramidy, který poprvé popularizoval Mike Cohn v knize *Succeeding with Agile*, definuje ideální distribuci různých typů automatizovaných testů [20]. Cílem je vytvořit robustní základnu rychlých a levných testů, které jsou doplněny menším množstvím komplexnějších a pomalejších scénářů.



Obrázek 1.3.: Model testovací pyramidy podle Mikea Cohna [20]

Pyramida se skládá ze tří hlavních úrovní:

- **Jednotkové testy (Unit Tests):** Tvoří základnu pyramidy. Testují jednotlivé funkce nebo třídy v izolaci od okolního prostředí (např. pomocí mockování). Jsou extrémně rychlé a poskytují vývojáři okamžitou zpětnou vazbu.

- **Integrační testy (Integration Tests):** Zaměřují se na interakci mezi různými moduly nebo externími službami, čímž ověřují správnou komunikaci mezi komponentami systému.
- **End-to-End testy (E2E):** Vrchol pyramidy. Simulují reálné chování uživatele a testují systém jako celek od vstupu v CLI až po vytvoření VPN tunelu v jádře. Jsou nejpomalejší a nejvíce náchylné k chybám v konfiguraci prostředí (tzv. flakiness).

Behavior-Driven Development (BDD) a Gherkin

Vývoj řízený chováním (BDD – Behavior-Driven Development) je metodika, která rozšiřuje principy TDD o zaměření na uživatelský pohled a čitelnost scénářů pro netechnické členy týmu [3]. Specifikace jsou psány v přirozeném jazyce pomocí syntaxe **Gherkin**.

Scénáře v Gherkinu využívají klíčová slova:

- **Given (Za předpokladu):** Definice výchozího stavu systému.
- **When (Když):** Akce, kterou uživatel nebo systém provede.
- **Then (Pak):** Očekávaný výsledek operace.

Tento přístup umožňuje vytvořit tzv. „živou dokumentaci“, která je zároveň spustitelným testem. V rámci moderního inženýrství jsou Gherkin scénáře často využity pro precizní definici požadavků na rozhraní a chování systému, což zajišťuje, že implementace přesně odpovídá navržené specifikaci.

1.6. Distribuce softwaru a balíčkovací systémy

Správa a distribuce softwaru v ekosystému Linuxu představuje komplexní proces, který zajišťuje integritu, bezpečnost a snadnou údržbu systému. Na rozdíl od monolitických operačních systémů, kde je instalace softwaru často v režii jednotlivých aplikací, Linux využívá centralizované balíčkovací systémy, které spravují software jako soubor vzájemně propojených komponent.

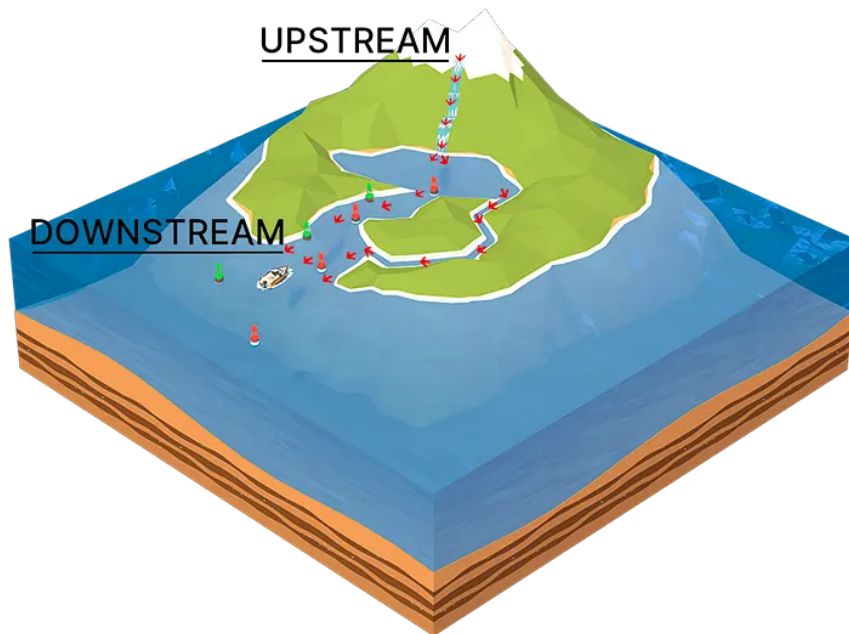
Hierarchie distribucí a model Upstream/Downstream

Linuxový ekosystém je charakteristický svou fragmentací, která je však organizována do logických struktur. Základem jsou tzv. foundational distributions (např. Debian, Fedora, Arch Linux, Slackware), ze kterých se odvozují stovky dalších variant.

Vztah mezi vývojáři a uživateli je definován modelem Upstream/Downstream [21]:

- **Upstream (Horní proud):** Představuje autory původního zdrojového kódu (např. projekt Linux kernel, GNOME, nebo samotný vývojář aplikace). Zde probíhá primární vývoj a opravy chyb.

- **Downstream (Dolní proud):** Představuje linuxové distribuce, které přebírají kód z upstreamu, upravují jej pro své potřeby (např. aplikací specifických patchů), kompilují a balí do binárních balíčků.



Obrázek 1.4.: Model Upstream a Downstream v distribuci softwaru [21]

Příkladem hierarchie je rodina Debian: Debian (foundational) → Ubuntu (downstream vůči Debianu) → Linux Mint (downstream vůči Ubuntu). Tato hierarchie umožňuje efektivní sdílení oprav a inovací – oprava v upstreamu postupně „proteče“ všemi downstream distribucemi k uživatelům.

Koncepty správy balíčků

Moderní správa balíčků je založena na třech pilířích:

1. **Metadata:** Každý balíček obsahuje podrobné informace o své verzi, architektuře, popisu a především o svých závislostech.
2. **Závislosti (Dependencies):** Schopnost systému automaticky identifikovat a doinstalovat knihovny nebo jiné programy, které balíček vyžaduje ke svému běhu. Tím se předchází tzv. „dependency hell“, kdy uživatel musel ručně dohledávat potřebné komponenty.
3. **Repozitáře:** Centralizovaná úložiště spravovaná distribucí, která slouží jako jediný důvěryhodný zdroj softwaru. To radikálně zvyšuje bezpečnost oproti stahování instalátorů z neznámých webových stránek.

Ekosystém Debian (.deb)

Formát balíčků `.deb` je standardem pro rodinu Debian. Teoretický základ Debian packagingu definuje Debian Policy Manual [22]. Balíček se skládá ze dvou hlavních částí: datového archivu (obsahujícího samotné binární soubory a dokumentaci) a kontrolního archivu.

Klíčovým prvkem je adresář `debian/`, který obsahuje:

- **control:** Definuje metadata jako název, verzi, sekci a pole **Depends** pro runtime závislosti a **Build-Depends** pro nástroje potřebné ke kompilaci.
- **rules:** Prováděcí skript (typicky Makefile), který definuje, jak se má software kompilovat a instalovat do dočasného adresáře před zabalením.
- *Maintainer scripts:* Skripty (`preinst`, `postinst`, `prerm`, `postrm`), které jádro systému spouští před instalací nebo po ní pro konfiguraci systémových služeb.

Ekosystém Red Hat a Fedora (.rpm)

Ecosystem založený na RPM (Red Hat Package Manager) je de facto standardem pro podnikovou sféru (RHEL) a komunitní distribuci Fedora. Standardy pro tuto rodinu definují Fedora Packaging Guidelines [23].

Zatímco Debian využívá sadu souborů v adresáři `debian/`, RPM staví na konceptu jediného Spec souboru (`.spec`). Tento soubor je deklarativním popisem celého životního cyklu balíčku:

- **%prep:** Sekce pro rozbalení zdrojového kódu a aplikaci záplat.
- **%build:** Instrukce pro kompilaci.
- **%install:** Definice instalace do virtuálního kořene (BuildRoot).
- **%files:** Explicitní seznam souborů, které má finální balíček obsahovat. Jakýkoliv soubor vytvořený v sekci `%install`, který není uveden v `%files`, způsobí chybu buildu, což zajišťuje vysokou úroveň kontroly nad obsahem balíčku.

Významným rozdílem je také práce se závislostmi. Zatímco rodina Debian využívá nástroj APT, RPM systémy přešly od YUM k modernímu nástroji DNF, který využívá pokročilé algoritmy (SAT solver) pro řešení konfliktů mezi balíčky.

1.7. Agilní metodiky a Extreme Programming

Moderní vývoj softwaru se v posledních dvou dekadách posunul od rigidních modelů typu vodopádový model (Waterfall) k agilním přístupům, které kladou důraz na adaptabilitu a lidský faktor. Jednou z nejvlivnějších metodik v této oblasti je Extreme Programming (XP), kterou definoval Kent Beck na konci 90. let 20. století. XP se zaměřuje na zvyšování kvality softwaru a schopnost týmu reagovat na měnící se požadavky zákazníka skrze intenzivní aplikaci osvědčených inženýrských praktik [24].

Filozofie a hodnoty XP

Základem metodiky XP je pět core hodnot, které tvoří etický a profesní rámec pro veškeré rozhodování v projektu [24]:

- **Komunikace (Communication):** Většina problémů v projektech pramení z nedostatku informací. XP podporuje přímou a otevřenou komunikaci mezi členy týmu i zákazníkem.
- **Jednoduchost (Simplicity):** Cílem je vytvořit „nejjednodušší věc, která může fungovat“. Tím se minimalizuje budoucí režie a technický dluh.
- **Zpětná vazba (Feedback):** Neustálé ověřování stavu systému skrze testy a interakci se zákazníkem umožňuje včasnou korekci kurzu.
- **Odvaha (Courage):** Schopnost provádět radikální změny v návrhu (refactoring) nebo přiznat chybu a zahodit nefunkční kód.
- **Respekt (Respect):** Vzájemná úcta mezi vývojáři a důvěra v jejich schopnosti jsou nezbytné pro fungování týmových praktik.

Klíčové praktiky XP

Hodnoty jsou v XP přetaveny do konkrétních technických a procesních praktik [24]. Pro tuto bakalářskou práci byly klíčové zejména:

- **Test-Driven Development (TDD):** Psaní testů před samotným kódem zajišťuje vysoké pokrytí, definuje očekávané chování a slouží jako pojistka při refactoringu.
- **Kontinuální integrace (Continuous Integration - CI):** Časté začleňování změn do hlavní větve projektu a automatické spouštění testů pro okamžitou detekci regresí.
- **Refactoring:** Neustálé vylepšování vnitřní struktury kódu bez změny jeho vnějšího chování s cílem zachovat čistotu a udržitelnost designu.
- **Jednoduchý návrh (Simple Design):** Návrh systému odpovídá pouze aktuálním požadavkům, čímž se předchází nadměrné komplexitě (over-engineering).
- **Párové programování (Pair Programming):** Praktika, kdy dva vývojáři pracují na jednom úkolu u jednoho počítače, což zvyšuje kvalitu kódu skrze okamžité code review.

Zatímco metodika XP je primárně navržena pro týmovou spolupráci (zejména párové programování), její technické praktiky jako TDD, CI a Refactoring jsou plně aplikovatelné i pro samostatně pracujícího vývojáře a byly pro tuto bakalářskou práci klíčové.

2. Analýza a specifikace požadavků

Cílem této kapitoly je analyzovat potřeby cílové skupiny uživatelů a na jejich základě definovat funkční i nefunkční požadavky na výslednou aplikaci. Součástí je také analýza možných architektonických přístupů.

Celý proces analýzy a sběru požadavků probíhal v agilním prostředí. Během vývoje se využívaly iterativní schůzky, pravidelné standupy a dedikované brainstormové sekce. Na těchto setkáních byli přítomni zkušení vývojáři a technický ředitel (CTO), kteří měli přímý kontakt se zákazníky a znali jejich konkrétní požadavky a zpětnou vazbu. Díky tomu bylo možné rychle a efektivně reagovat na potřeby uživatelů a správně specifikovat klíčové vlastnosti aplikace.

2.1. Identifikace zúčastněných stran

Primárními uživateli GoodAccess CLI pro Linux jsou:

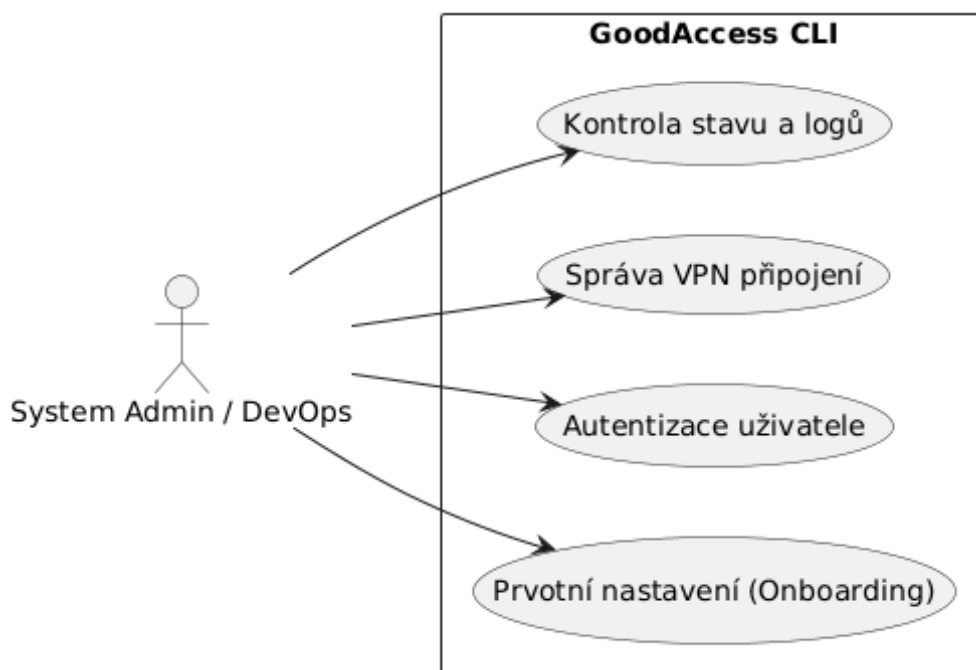
- **System Administrators:** Spravují servery a vyžadují stabilní nástroj pro vzdálenou správu přes SSH bez nutnosti grafického rozhraní.
- **DevOps Inženýři:** Potřebují integrovat VPN připojení do automatizovaných skriptů a CI/CD pipeline. Vyžadují predikovatelné chování a strojově čitelný výstup.
- **Běžní zaměstnanci:** Uživatelé, kteří preferují textové uživatelské rozhraní (TUI) před grafickým klientem (GUI) pro každodenní práci.

2.2. Případy užití

Hlavní scénáře použití systému z pohledu koncového uživatele (administrátora či běžného zaměstnance) ukazuje následující diagram případů užití (Obrázek 2.1).

2.3. Funkční požadavky

Na základě analýzy potřeb byly stanoveny následující funkční požadavky:



Obrázek 2.1.: Diagram případů užití GoodAccess CLI

Autentizace a správa relace

Aplikace musí umožnit přihlášení uživatele v prostředí bez webového prohlížeče. Z tohoto důvodu nepodporuje přihlášení pomocí webového SSO (Single Sign-On).

- **F1 – Login:** Přihlášení probíhá pomocí standardního terminálového vstupu, kam uživatel zadá název týmu, uživatelské jméno a heslo. Zadávání hesla je pro bezpečnost maskováno (zobrazují se pouze znaky *).
- **F2 – Logout:** Standardní ukončení relace uživatele a bezpečné smazání lokálně uložených tokenů.

Správa připojení

Jádrem aplikace je ovládání VPN tunelu a správa nastavení pro připojení.

- **F3 – Connect:** Navázání připojení. Standardně se využije preferovaný protokol a brána (gateway) uložená z předchozího nastavení.
- **F4 – Connect Flags:** Možnost dočasně přepsat preferované nastavení při připojení pomocí přepínačů **-g** (gateway) a **-p** (protocol). Přepínač **--protocol** očekává textový název protokolu, zatímco **--gateway** vyvolá interaktivní výběr. Tato volba se neuloží jako preferovaná.
- **F5 – Disconnect:** Korektní ukončení VPN připojení a úklid routovacích pravidel.
- **F6 – Persistent Connect:** Nastavení připojení na úrovni celého zařízení, které zajistí automatické znovupřipojení i po restartu počítače.

- **F7 – Gateway Selection:** Možnost uživatele vybrat si konkrétní VPN bránu.
- **F8 – Protocol Selection:** Možnost uživatele zvolit preferovaný VPN protokol (WireGuard nebo OpenVPN).

Monitoring a stav

- **F9 – Status:** Zobrazení aktuálního stavu připojení (Připojeno/Odpojeno, IP adresa, přenesená data). Výstup lze volitelně formátovat do JSON pro snadné strojové zpracování.

Správa konfigurace a konfliktů

- **F10 – Setup:** Interaktivní průvodce prvotním nastavením (onboarding wizard). Umožňuje uživateli v jednom kroku provést přihlášení, zvolit protokol, vybrat bránu, nastavit perzistentní připojení a rovnou se připojit. Tento proces je striktně interaktivní.
- **F11 – Řešení konfliktů (Conflict Resolution):** Systém musí detekovat konfliktní stavy. Pokud je aktivní relace v grafickém klientovi (GUI) nebo je na stejném stroji připojen jiný uživatel, CLI klient na to uživatele upozorní a striktně zamezí přepisu stavu sítě.

Obecné příkazy

- **F12 – Version:** Zobrazení aktuální verze aplikace.
- **F13 – Help:** Zobrazení standardní nápovědy k dostupným příkazům a jejich použití.

2.4. Nefunkční požadavky

- **N1 – Výkon a odezva:** Aplikace v prostředí příkazové řádky (CLI) musí reagovat okamžitě. Zásadní je rychlý start aplikace a nízká paměťová náročnost, aby její spuštění uživatele neomezovalo.
- **N2 – Kompatibilita:** Aplikace musí být funkční na hlavních serverových distribucích Linuxu (Debian/Ubuntu, Fedora/RHEL, Arch Linux).
- **N3 – Bezpečnost:** Citlivá data (přístupové tokeny, privátní klíče) musí být uložena bezpečně a přístupná pouze oprávněné službě.
- **N4 – Distribuce a nasazení:** Pro usnadnění instalace a správy ze strany systémových administrátorů musí být aplikace zabalitelná do standardních distribučních balíčků. Konkrétně se jedná o formáty `.deb` (pro rodinu Debian) a `.rpm` (pro rodinu Red Hat).

2.5. Analýza architektury

Analýza stávajícího řešení GoodAccess

Aplikace GoodAccess na operačním systému Linux v současné době funguje jako rozdělený systém složený ze dvou hlavních komponent:

- **GoodAccessService (Daemon):** Tato služba běží na pozadí pod správou systému `systemd` s právy uživatele `root`. Její primární odpovědností je správa síťových rozhraní (především tunelů WireGuard), konfigurace systémového směrování (routingu) a bezpečné uchovávání kryptografických klíčů a certifikátů.
- **Grafický klient (Electron/GUI):** Uživatelské rozhraní je postaveno na frameworku Electron. Tato část běží v běžném uživatelském prostoru (bez root oprávnění) a funguje v podstatě jako „hloupý“ prezentační ovladač. Zobrazuje stav a přijímá příkazy od uživatele, které následně předává daemonu ke zpracování. Komunikace mezi těmito dvěma procesy probíhá lokálně, typicky pomocí Unix Domain Sockets (IPC).

Zhodnocení problému a motivace pro CLI

Tato rozdělená architektura funguje spolehlivě na běžných desktopových instalacích Linuxu. Zásadním problémem stávajícího řešení je však závislost grafického klienta na frameworku Electron. Aplikace v Electronu pro své spuštění nutně vyžaduje přítomnost zobrazovacího serveru (např. X11 nebo Wayland).

Na serverových (headless) distribucích, v prostředích pro kontinuální integraci (CI/CD) nebo uvnitř Docker kontejnerů tyto zobrazovací servery záměrně chybí. V takových případech je spuštění a ovládání grafické aplikace nemožné, přestože klíčová běhová služba (daemon) dokáže fungovat bez problému na pozadí. Z tohoto omezení vyvstává explicitní potřeba vytvořit alternativní rozhraní ve formě aplikace pro příkazovou řádku (CLI), které dokáže plnohodnotně komunikovat s existující službou a ovládat ji čistě v textovém režimu, nezávisle na grafickém prostředí.

Možné architektonické přístupy

Při návrhu řešení byly zvažovány dva přístupy k interakci s existující logikou GoodAccess:

Návrh A: Tenký klient (Wrapper)

V tomto modelu běží v systému jedna privilegovaná služba (.NET), která drží stav a vykonává operace. CLI aplikace (Go) slouží pouze jako dálkové ovládání, které posílá příkazy službě přes IPC.

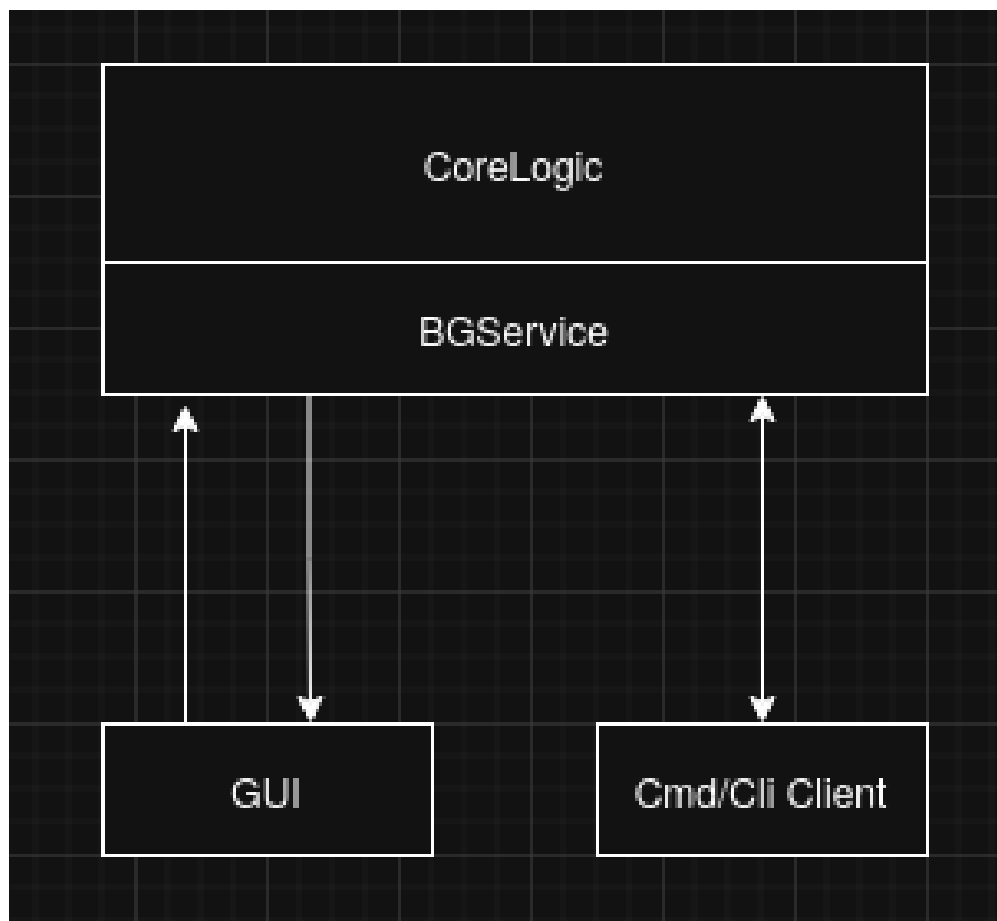
Návrh B: Samostatná logika

CLI aplikace by obsahovala veškerou logiku pro připojení a správu WireGuardu a běžela by přímo pod uživatelem root, nezávisle na existující službě.

Zvolené řešení

Byl zvolen **Návrh A (Wrapper)** z následujících důvodů:

1. **Single Source of Truth:** Služba drží jednotný stav aplikace. Nedochozí k desynchronizaci mezi tím, co si myslí CLI, a tím, co je reálně nastaveno v síti.
2. **Bezpečnost:** Uživatel nemusí spouštět CLI pod rootem. Privilegované operace provádí pouze izolovaná služba na pozadí.
3. **Znovupoužitelnost:** Využívá se již otestovaná a existující logika v .NET backendu, což je v souladu s principy XP (Simplicity).



Obrázek 2.2.: Zvolená architektura: Tenký klient (Wrapper)

3. Návrh řešení

V této kapitole je představen detailní technický návrh aplikace. Vychází z analytických požadavků a zvolené architektury tenkého klienta.

3.1. Celková architektura systému

Jak bylo podrobně rozebráno v předchozí kapitole, během analýzy byly zvažovány dvě hlavní cesty: vytvoření tenkého klienta napojeného na existující .NET službu, nebo vývoj zcela nezávislé aplikace napsané čistě v jazyce Go (Návrh B).

Zvolené řešení: Tenký klient (Wrapper)

Na základě zhodnocení byl pro finální implementaci CLI zvolen první přístup, tedy **tenký klient (Návrh A)**. V tomto modelu zůstává plně zachována existující privilegovaná služba v .NET, která bezpečně drží stav a vykonává veškeré síťové operace. Nová CLI aplikace, napsaná v jazyce Go, pak funguje výhradně jako interaktivní dálkové ovládání, které si předává zprávy se službou prostřednictvím lokálního IPC mechanismu.

Tento návrh je postaven na striktním oddělení kompetencí (Separation of Concerns). Architektura je rozdělena na dvě klíčové komponenty:

- **Služba (Service - .NET):** Funguje jako daemon spravovaný systémem `systemd` s právy uživatele `root`. Její primární a jedinou odpovědností je správa stavu sítě, konfigurace rozhraní WireGuard a bezpečné ukládání kryptografických klíčů. Volba platformy .NET pro tuto komponentu přímo vychází z potřeby multiplatformní podpory a snadné integrace do systému `systemd` (viz sekce 1.4), a především umožňuje znovupoužití již existující a otestované logiky z grafického klienta (viz kapitola 2).
- **Klient (Client - Go):** Běží v uživatelském prostoru (bez `root` oprávnění) a zajišťuje čistě interakci s uživatelem. Zodpovídá za parsování argumentů příkazové řádky pomocí frameworku `Cobra`, obsluhu interaktivních TUI prvků prostřednictvím `Bubble Tea` a finální formátování výstupu. Jazyk Go byl pro vývoj CLI zvolen díky svým vlastnostem pro systémové programování, rychlému startu a statické kompilaci do jediné binárky, což radikálně zjednodušuje distribuci (viz sekce 1.4 a 1.1).

Architektura Go klienta navíc úzce reflektuje principy takzvané *Clean Architecture*. Celá kódová základna klienta je rozdělena do logických vrstev, kde uživatelské rozhraní (UI a parsování příkazů) je zcela odděleno od aplikační logiky a transportní vrstvy (IPC komunikace přes Unix Domain Sockets). Toto vrstvení zaručuje vysokou testovatelnost komponent a možnost nezávisle upravovat vizuální prezentaci bez vlivu na způsob, jakým klient komunikuje s .NET službou.

Tento přístup, namísto budování celé logiky v Go, byl zvolen a potvrzen z následujících důvodů:

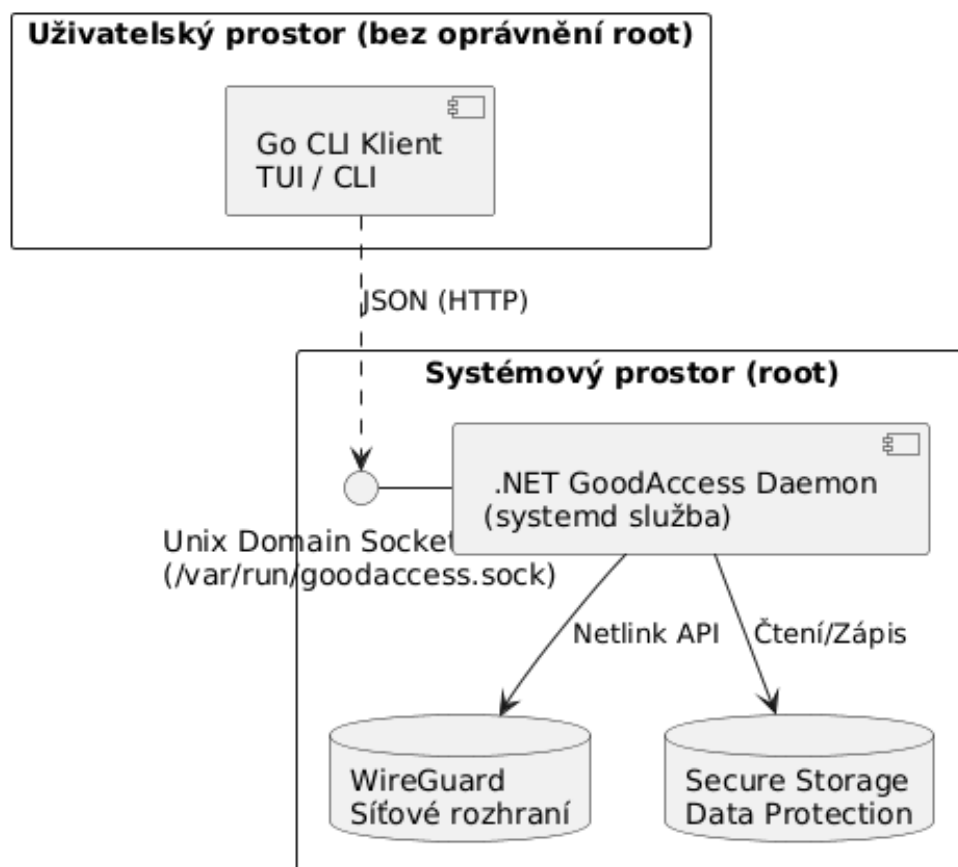
1. **Single Source of Truth:** Služba drží jednotný stav aplikace. Nedochází k desynchronizaci mezi tím, co si myslí CLI, a tím, co je reálně nastaveno v síti.
2. **Bezpečnost:** Uživatel nemusí spouštět CLI pod `rootem`. Privilegované operace provádí pouze izolovaná služba na pozadí.
3. **Znovupoužitelnost:** Využívá se již otestovaná a existující logika v .NET backendu, což je v souladu s agilními principy.

3.2. Návrh komunikace (IPC)

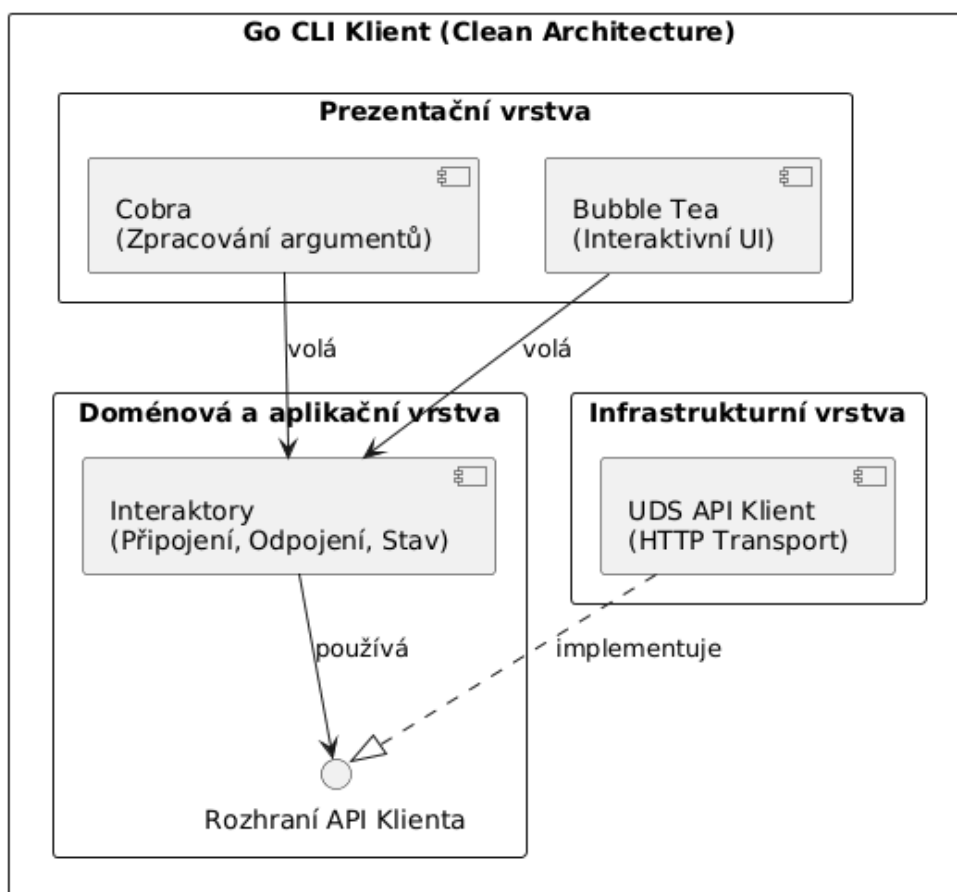
3.3. Návrh uživatelského rozhraní (CLI)

3.4. Bezpečnostní model

3.5. Datový model a konfigurace



Obrázek 3.1.: Architektura systému znázorňující hranici mezi .NET Službou a Go Klientem skrze IPC (Unix Domain Sockets).



Obrázek 3.2.: Struktura vrstev Go klienta v souladu s principy Clean Architecture.

4. Implementace

Kapitola popisuje proces vývoje, použitou strukturu projektu a klíčové implementační detaily jednotlivých komponent.

4.1. Struktura projektu a nástroje

4.2. Implementace klienta (Frontend)

4.3. Úpravy backendové služby

4.4. Integrace WireGuardu

4.5. Distribuční balíčky a Systemd

5. Testování a validace

Ověření funkčnosti a stability aplikace probíhalo v souladu s metodikou Extrémního programování průběžně během vývoje.

5.1. Metodika testování

5.2. Jednotkové testy (Unit Testing)

5.3. Integrační a systémové testy

5.4. Testování na různých distribucích

Závěr

Cílem této bakalářské práce byl návrh a implementace nativního CLI klienta GoodAccess pro OS Linux.

Shrnutí výsledků

Diskuse a přínos

Možnosti budoucího rozvoje

Seznam použitých zdrojů

1. RAYMOND, Eric Steven. *The Art of Unix Programming*. Addison-Wesley Professional, 2003. ISBN 978-0-13-142901-7.
2. WARD, Brian. *How Linux Works: What Every Superuser Should Know*. 3rd. No Starch Press, 2021. ISBN 978-1-7185-0040-2.
3. CHARMBRACELET. *Bubble Tea: A powerful little TUI framework* [online]. 2024. [cit. 2026-03-02]. Dostupné z: <https://github.com/charmbracelet/bubbletea>.
4. DONENFELD, Jason A. WireGuard: Next Generation Kernel Network Tunnel. In: *NDSS 2017: Network and Distributed System Security Symposium*. 2017. Dostupné také z: <https://www.wireguard.com/papers/wireguard.pdf>.
5. CLOUDFLARE. *Castle-and-moat network security* [online]. 2024. [cit. 2026-03-02]. Dostupné z: <https://www.cloudflare.com/learning/access-management/castle-and-moat-network-security/>.
6. NIST. *Zero Trust Architecture (SP 800-207)* [online]. 2020. [cit. 2026-03-02]. Dostupné z: <https://csrc.nist.gov/publications/detail/sp/800-207/final>.
7. CLOUDFLARE. *What is Zero Trust?* [online]. 2024. [cit. 2026-03-02]. Dostupné z: <https://www.cloudflare.com/learning/security/glossary/what-is-zero-trust/>.
8. GOODACCESS. *GoodAccess Documentation* [online]. 2024. [cit. 2024-11-07]. Dostupné z: <https://support.goodaccess.com/>.
9. GOODACCESS. *2. Architecture Overview* [online]. 2024. [cit. 2026-02-12]. Dostupné z: <https://support.goodaccess.com/getting-started/2.-architecture-overview>.
10. PRESS, GSC Online. *Zero Trust Network Access: A Review* [online]. 2025 [cit. 2026-03-02]. Dostupné z: <https://gsconlinepress.com/journals/gscarr/sites/default/files/GSCARR-2025-0036.pdf>.
11. PRICE, Mark J. *C# 12 and .NET 8 – Modern Cross-Platform Development Fundamentals*. 8th. Packt Publishing, 2023. ISBN 978-1-83763-587-0.
12. MICROSOFT. *.NET Generic Host* [online]. 2024. [cit. 2026-03-02]. Dostupné z: <https://learn.microsoft.com/en-us/dotnet/core/extensions/generic-host>.

13. MICROSOFT. *ASP.NET Core Data Protection Overview* [online]. 2024. [cit. 2026-03-02]. Dostupné z: <https://learn.microsoft.com/en-us/aspnet/core/security/data-protection/introduction>.
14. GO AUTHORS. *The Go Programming Language Specification* [online]. 2024. [cit. 2026-03-02]. Dostupné z: <https://go.dev/ref/spec>.
15. DONOVAN, Alan A. A.; KERNIGHAN, Brian W. *The Go Programming Language*. Addison-Wesley Professional, 2015. ISBN 978-0-13-419044-0.
16. COBRA AUTHORS. *Philosophy of Cobra* [online]. 2024. [cit. 2026-03-02]. Dostupné z: <https://cobra.dev/docs/explanations/philosophy/>.
17. HASKELL, Andy. *Intro to Bubble Tea in Go* [online]. 2022. [cit. 2026-03-02]. Dostupné z: <https://dev.to/andyhaskell/intro-to-bubble-tea-in-go-21lg>.
18. BRAY, T. *The JavaScript Object Notation (JSON) Data Interchange Format (RFC 8259)* [online]. 2017. [cit. 2026-03-02]. Dostupné z: <https://datatracker.ietf.org/doc/html/rfc8259>.
19. IBM CLOUD EDUCATION. *gRPC vs. REST: What's the Difference?* [online]. 2024. [cit. 2026-03-02]. Dostupné z: <https://www.ibm.com/cloud/blog/grpc-vs-rest>.
20. COHN, Mike. *Succeeding with Agile: Software Development Using Scrum*. Addison-Wesley Professional, 2009. ISBN 978-0-321-57936-2.
21. LUMINOUSMEN. *Understanding Upstream and Downstream in Data Pipelines* [online]. 2023. [cit. 2026-03-02]. Dostupné z: <https://luminousmen.medium.com/data-engineering-terminology-understanding-upstream-and-downstream-in-data-pipelines-bd00381515b5>.
22. DEBIAN PROJECT. *Debian Policy Manual* [online]. 2024. [cit. 2024-11-07]. Dostupné z: <https://www.debian.org/doc/debian-policy/>.
23. FEDORA PROJECT. *Fedora Packaging Guidelines* [online]. 2024. [cit. 2026-02-18]. Dostupné z: <https://docs.fedoraproject.org/en-US/packaging-guidelines/>.
24. BECK, Kent; ANDRES, Cynthia. *Extreme Programming Explained: Embrace Change*. 2nd. Addison-Wesley, 2004. ISBN 978-0-321-27865-4.

A. Externí přílohy

Externí přílohy této bakalářské práce jsou umístěny na adrese:

https://github.com/Jiri-Fiser/thesis_ki_ujep.

Na úložišti GitHub mohou být uloženy tyto externí přílohy:

- **zdrojové kódy**
- **doplňkové texty** (například jak instalovat aplikaci, manuály aplikace)
- **schémata** (především, pokud se nevejdou na stranu A4 a jejich vytištění je tak problematické)
- **screenshoty** (v textu práce lze použít jen omezený počet snímků obrazovky, které navíc nemusí být při černobílém tisku příliš přehledné)
- **videa** (například ovládání aplikace)

V každém případě by to však měli být pouze materiály, které jste vytvořili sami. Materiály jiných autorů uvádějte v seznamu použité literatury (včetně případných odkazů na jejich originální umístění).

V této kapitole stačí uvést pouze základní strukturu úložiště (co se kde nalézá a jakou má funkci) například v podobě tabulky.

ki-thesis.pdf	text práce v PDF
ki-thesis.tex	zdrojový kód práce v L ^A T _E Xu
kitheses.cls	definice třídy dokumentů (rozšířená třída scrbook)
thesis.bib	bibliografická databáze (exportována z citace.com)
LOGO_PRF_CZ_RGB_standard.jpg	logo fakulty s českým textem
LOGO_PRF_EM_RGB_standard.jpg	logo fakulty s anglickým textem

Všechny tyto soubory jsou potřeba pro překlad dokumentu (logo stačí jedno v příslušné jazykové verzi).

B. Další přílohy

Výjimečně může práce obsahovat i další tištěné přílohy. Obecně však dávejte přednost elektronickým přílohám umístěným na GitHubu (tato kapitola tak bude úplně chybět).